

CyClaw

Offline-First, RAG-Enforced Soul Agent

Architecture Reference Document | v1.0.8 | June 2026

Chris Grady | @CGFixIT | github.com/CGFixIT/CyClaw | cgfixit.com

v1.0.8 Update Highlights

- **Agentic Layer (v0.1)** — read-only GitHub ops via *agentic.cli*; write scaffold (dry-run only); `pr_view`, `pr_list`, `pr_diff`, `issue_view`, `issue_list`, `repo_view`
- **Filesystem Connector (v0.1)** — scoped read/write to *allowed_roots*; content-safety scan; RAG index of file share; writes gate-disabled by default
- **SQL Connector scaffold (v0.1)** — read-only Postgres/MSSQL adapter; `schema_list`, `table_preview`, `run_select`; fully disabled by default
- **NeMo Guardrails skeleton** — input/output/topical rails (`check_injection`, `check_jailbreak`, `check_soul_mutation`, `check_grounding`, `check_soul_leak`); degrades to offline heuristics when nemoguardrails absent
- **Dropbox Corpus Sync** — rclone-based pull (bisync opt-in); safety fuses: `max_delete=20`, `max_transfer=1G`; auto-reindex on change
- **Prompt injection expanded to 33 patterns** — added memory/persistence poisoning, authority/urgency, tool/MCP hijacking, obfuscation categories; OWASP LLM01 + agent-security aligned
- **PII redaction hardened** — Bearer tokens + `api_key` forms added to audit-log redaction; closes gap where retrieval errors leaked tokens to `audit.jsonl`
- **Grok model updated** — `grok-beta` → `grok-4.3` (`grok-4` retired 2026-05-15); `grok-4.3` is current flagship
- **Rate limiter now config-persisted** — optional SQLite or Postgres backend for per-IP counters across restarts; in-memory default unchanged
- **Graph timeout hardened** — `asyncio.wait_for` wraps `compiled_graph.invoke`; HTTP 504 on exceed (prevents stalled LM Studio hanging requests)
- **TrustedHostMiddleware** — DNS-rebinding defense; Host header checked against `allowed_hosts` allowlist before any handler runs
- **pgvector backend (opt-in)** — ChromaDB stays default; install `cyclaw[pgvector]` extra to move to Postgres vector store
- **soul_max_chars cap** — 8000 char ceiling on soul preamble prevents oversized `soul.md` from blowing LM Studio context budget
- **All 5 security invariants preserved and structurally strengthened**

Property	Value
Version	1.0.8 (agentic layer + connectors + 33-pattern filter + NeMo skeleton)
Runtime	FastAPI 0.138 + LangGraph 1.2.6 + LM Studio (local GGUF)
Retrieval	ChromaDB 1.5.9 (vector) + BM25 (keyword) + RRF fusion; pgvector opt-in
Embeddings	sentence-transformers 5.6.0 / all-MiniLM-L6-v2 (CPU, 384-dim)

Property	Value
Soul Governance	SQLite + Markdown, versioned, atomic writes, drift-detected, 365-day TTL, 8000-char cap
Rate Limiting	Sliding window 60 req/min per IP; optional SQLite/Postgres persistence
Sanitizer	33 regex patterns (OWASP LLM01 + memory poisoning + tool hijack + obfuscation)
Fallback	Grok-4.3 via xAI API (user-gated, config-gated, env-gated)
MCP Server	Retrieval-only, sampling = null, no LLM access
Agentic Layer	Read-only GitHub ops (v0.1); write dry-run scaffold; fully out-of-band
FS Connector	Scoped read/write to allowed_roots; writes gate-disabled by default (v0.1)
Guardrails	NeMo skeleton (input/output/topical rails); degrades gracefully offline
Binding	127.0.0.1:8787 + TrustedHostMiddleware (DNS-rebinding defense)
Audit	SHA-256 hashed, PII-redacted (Bearer+api_key added), append-only JSONL
Browser UI	terminal.html (Soul Console, ~49 KB) + extractor.html
Sync	Dropbox corpus pull via rclone; safety fuses; auto-reindex on change

Version History

Version	Status	Notes
v1.2.0	Superseded	Baseline production release
v1.3.0	Superseded	Rate limiting (60/min), 13 OWASP patterns, soul SHA-256 drift detection, atomic writes, TTL → 365d
v1.4.0	Superseded	Python 3.12 hardening, constraints.txt, CVE patches, MCP test suite, /soul/restore endpoint
v1.5.0	Superseded	Stemmer fix, SQL write cleanup, Dropbox sync integration, BM25 SHA integrity check on load
v1.6.0	Superseded	Rate limiter config persistence, TrustedHostMiddleware, graph timeout (504), soul_max_chars cap, pgvector opt-in
v1.7.0	Superseded	PyPI package v1.7.0; cyclaw-server entry point; full uv/pyproject modernization
v1.0.8	Production (current)	Agentic layer, FS connector, SQL connector scaffold, NeMo guardrails, 33-pattern filter, grok-4.3, richer PII redaction

Table of Contents

- A. System Overview — High-level topology, client interfaces, layer stack
- B. Gateway Layer — FastAPI endpoints, rate limiter, CORS, prompt filter, GraphState
- C. LangGraph Controller — 7-node directed graph, routing logic, soul injection, convergence
- D. Retrieval Layer — Corpus indexing, hybrid search, RRF fusion, stemmer
- E. Soul Governance — Drift detection, atomic writes, TTL prune, soul_max_chars cap
- F. Model Layer — LM Studio, sentence-transformers, Grok-4.3 fallback, triple gate
- G. MCP Server — Retrieval-only tool exposure, protocol constraints
- H. Security & Sanitization — 33-pattern filter, PII redaction, TrustedHostMiddleware, rate limiting
- I. Browser UI — terminal.html Soul Console (~49 KB), extractor.html corpus tool
- J. Agentic Layer — Read-only GitHub ops, write scaffold (dry-run), registry, isolation

- K. Connectors** — Filesystem connector, SQL connector, Dropbox corpus sync
- L. NeMo Guardrails** — Input/output/topical rails, offline heuristic fallback
- M. Test Suite** — Full pytest coverage + PowerShell smoke + CI gates
- N. Deployment** — v1.0.8 upgrade path, install, pre-flight, validation, rollback
- O. v1.0.8 Change Log** — Full diff summary against v1.4.0 baseline
- P. Security Invariants** — 5 topology-enforced guarantees (preserved + structurally strengthened)

A. System Overview

CyClaw is a seven-layer architecture (gateway, controller, retrieval, soul, models, MCP, agentic/connectors) where each layer has a single responsibility and explicit boundaries. The LangGraph controller is the security enforcement mechanism — routing decisions are made by graph topology, not by prompt instructions. v1.0.8 adds an out-of-band agentic layer, filesystem and SQL connectors, NeMo guardrails skeleton, and Dropbox corpus sync, all of which are strictly isolated: none are imported by gate.py, graph.py, or mcp_hybrid_server.py.

Client Interfaces

Interface	Protocol	Entry Point
CLI / Scripts	HTTP	POST /query, /health, /soul/*
Browser UI (terminal.html)	HTTP	/ (StaticFiles) → /query, /soul/*
Extractor (extractor.html)	HTTP	Corpus extraction tool
LM Studio / Claude Desktop	MCP (stdio)	mcp_hybrid_server.py
Agentic CLI	subprocess/gh	python -m agentic.cli (never gate.py)
FS Connector CLI	filesystem	python -m agentic.fsconnect.cli
SQL Connector CLI	ODBC/psycopg	python -m agentic.sqlconnect.cli
Dropbox Sync CLI	rclone	python -m sync.cli

Layer Stack

Layer	Module(s)	Responsibility
A. Gateway	gate.py, mcp_hybrid_server.py	Rate limit, sanitization, CORS, TrustedHostMiddleware, static serving, GraphState init
B. Controller	graph.py	7-node LangGraph: retrieve → route → respond → audit
C. Retrieval	retrieval/*.py	Corpus indexing, hybrid search (Chroma + BM25 + RRF); pgvector opt-in
D. Soul	utils/personality.py, data/personality/	Versioned identity, drift-detected, atomic writes, TTL pruning, soul_max_chars cap
E. Models	llm/client.py	LM Studio (local), sentence-transformers, Grok-4.3 (triple-gated)
F. MCP Server	mcp_hybrid_server.py	Retrieval-only tool; sampling = null
G. Agentic	agentic/cli.py, agentic/fsconnect/, agentic/sqlconnect/	Out-of-band GitHub ops + FS + SQL connectors; never imported by gateway
H. Sync	sync/cli.py	rclone-based Dropbox corpus pull; out-of-band only

Layer	Module(s)	Responsibility
I. Guardrails	guardrails/cli.py	NeMo rails skeleton; out-of-band; degrades offline gracefully

B. Gateway Layer

The FastAPI gateway (gate.py) and MCP hybrid server are the only external interfaces. Both bind to 127.0.0.1 — no external network exposure by default. v1.0.8 adds TrustedHostMiddleware (DNS-rebinding defense), configurable rate-limit persistence (SQLite/Postgres), graph-level timeout wrapped in asyncio.wait_for, and a hardened error sanitizer that strips Bearer tokens and api_key patterns from HTTP error bodies.

HTTP Endpoints

Endpoint	Method	Description
/query	POST	Main entry — rate-limit check, TrustedHost check, sanitize, build GraphState, invoke graph (504 on timeout)
/health	GET	Liveness + dependency health checks (LM Studio, Grok, embeddings)
/soul	GET	Current soul content, version, source path
/soul/propose	POST	Diff preview with SHA hashes — does NOT apply
/soul/apply	POST	Applies evolution — requires human reason string
/soul/reload	POST	Reload soul.md from disk after manual edit
/soul/restore	POST	Restores soul.md from .bak backup
/audit/summary	GET	Aggregated audit metrics (API-key gated; Bearer token)
/ops/sync	POST	Trigger Dropbox corpus sync (API-key gated, out-of-band subprocess)
/ops/agentic	POST	Trigger agentic read op (API-key gated, out-of-band subprocess)
/	GET	Serves terminal.html (Soul Console) via StaticFiles mount

Gateway Invariants

- TrustedHostMiddleware runs first — requests with disallowed Host headers get HTTP 400 before any handler
- Rate-limit check runs before any work — exhausted clients hit 429; optional SQLite/Postgres persistence (v1.0.8)
- Banned-pattern prompt filter (33 patterns) runs before any LLM call
- Max input size 4000 chars enforced at gateway level
- CORS restricted to security.allowed_origins (config-driven allowlist)
- GraphState initialized with: query, user_confirmed_online
- All responses follow uniform JSON schema (answer, sources, model_used, retrieval_mode, needs_confirm, error)
- graph_timeout_sec (330s default) wraps compiled_graph.invoke — HTTP 504 on exceed, no stalled workers
- Secrets redaction: Bearer tokens + api_key patterns stripped from error bodies before HTTP response
- Telemetry kill-switch env vars set at module top, BEFORE any langchain/chromadb import (10 vars verified at startup)

Rate Limiter (v1.0.8 — persistence-capable)

```
# In-memory default (persist_path=null, database_url=null) _rate_limits = defaultdict(list) RATE_LIMIT_REQUESTS = 60 # per-IP RATE_LIMIT_WINDOW = 60 # seconds # Optional: persist_path = 'data/rate_limits.db' → SQLite survival across restarts # Optional: database_url = 'postgres://...' → Postgres (takes precedence)
```

C. LangGraph Controller

The controller is a 7-node directed graph where **topology IS the security policy**. Routing decisions are structural — enforced by graph edges, not by prompt wording. Every execution path terminates at the `audit_logger` node before returning to the gateway. Soul injection happens at the prompt construction level inside `local_llm` and `offline_best_effort` — there is no soul graph node. Evolution is an explicit HTTP endpoint, not a graph operation. All nodes are partial functions injected at build time, making the graph stateless and safe to reuse across requests.

Graph Topology (7-node directed)

```
[ENTRY] | 1. retrieve (Chroma + BM25 + RRF -> retrieved_docs, top_score) | 2. route_by_score (top_score >=
cfg.retrieval.min_score [0.028 RRF]?) | score >= threshold |-> 3. local_llm (soul + query + docs -> LM Studio) |
score < threshold |-> 4. user_gate (sets needs_user_confirm; awaits resubmit) | user=yes + hybrid + grok.enabled
|-> 5. grok_fallback | user=no OR offline mode |-> 6. offline_best_effort | 7. audit_logger (SHA-256 query hash +
PII redact -> logs/audit.jsonl) -> END
```

All paths converge at `audit_logger`. Convergence is enforced by graph edges, not code discipline — there is no shortcut path. Personality interactions are recorded to the shadow DB after audit so soul-level history stays consistent with the audit trail. The graph is compiled once at startup with no persistent checkpointer (every request is stateless).

Node Descriptions

Node	Inputs	Outputs	Notes
retrieve	query	retrieved_docs, top_score, retrieval_mode	Always first; degrades gracefully to semantic-only or BM25-only
route_by_score	top_score	Conditional edge	Graph topology, not LLM decision; 0.028 RRF threshold
local_llm	soul_core, query, retrieved_docs	answer, answer_model, answer_sources	LM Studio (qwen2.5-7b-instruct); soul injected as preamble; context budget enforced
user_gate	top_score (below threshold)	needs_user_confirm=True	Pauses; client must resubmit with user_confirmed_online
grok_fallback	user_confirmed_online=True + gates	answer, answer_model	Triple-gated; local context NOT forwarded by default
offline_best_effort	Any non-Grok confirmed path	answer, answer_model	Best-effort local; soul-injected; used when user declines Grok
audit_logger	Full GraphState	audit_event → JSONL	SHA-256 hash + PII redaction (Bearer+api_key added v1.0.8); always terminal

GraphState Schema

Field	Type	Set By
query	str	Gateway (from user input)
user_confirmed_online	bool None	Gateway (from client resubmit)
retrieved_docs	list[RetrievedDoc]	retrieve node
top_score	float	retrieve node
retrieval_mode	str	retrieve node (hybrid semantic bm25 none)

Field	Type	Set By
needs_user_confirm	bool	route_by_score / user_gate
answer	str	local_llm grok_fallback offline_best_effort
answer_model	str	Responding node (local grok offline-best-effort)
answer_sources	list[RetrievedDoc]	Responding node
audit_event	dict	audit_logger node
error	str None	Any node on failure

D. Retrieval Layer

Retrieval is the unconditional first node in the graph. The hybrid retriever combines semantic (ChromaDB cosine HNSW) and keyword (BM25Okapi) search via Reciprocal Rank Fusion (RRF). If either path fails, the retriever degrades gracefully to whichever path remains viable. Both paths return normalized scores: the semantic-only fallback returns raw cosine similarity; the BM25-only fallback is rebased into RRF range via `_normalize_single_path()` so `min_score` comparisons stay consistent.

Indexing Pipeline (offline batch)

- **load_corpus()** — recursive read of `.md/.txt` from `data/corpus/` (extension match is case-insensitive; symlinks escaping corpus root are rejected)
- **chunk_document(size=512, overlap=50)** with sentence-boundary snapping
- **sanitize_chunk()** — strips banned patterns at ingestion (33 patterns in v1.0.8)
- **tokenize_and_stem()** — enhanced Porter stemmer (technical-vocab safe; custom AI/DevOps terms)
- **embed_chunks()** — 384-dim via all-MiniLM-L6-v2 (CPU, batch=32); cached in `.emb_cache/`
- **Vector backend write** — Chroma PersistentClient → `index/chroma_db/` (default) OR `pgvector` (opt-in via `indexing.vector_backend: pgvector`)
- **Build BM25 index** → `index/bm25.json` (JSON: `tokenized_corpus` + `chunks` + `metadata`; pickle removed for security)

Query-Time Hybrid Search

Component	Method	Effective Weight
Semantic	Chroma vector search (embed → cosine sim) or pgvector	1.0 (equal in RRF)
Keyword	BM25 (tokenize + stem → term frequency)	1.0 (equal in RRF)
Fusion	RRF (k=60): $\text{score} = \sum 1/(k + \text{rank}_i)$	—

Each `SearchResult` carries full RRF provenance: `semantic_score`, `semantic_rank`, `keyword_score`, `keyword_rank`, `rrf_score`, `rrf_semantic_contrib`, `rrf_keyword_contrib`. All fields are exposed in the `SourceInfo` response model. **Design note:** CyClaw uses equal-weighted RRF. The config keys `vector_weight/bm25_weight` are documentation-only placeholders — not multiplied into RRF contributions. To opt into weighted RRF, multiply `contrib *= weight` in `hybrid_search.py` AND retune `retrieval.min_score`.

Retrieval Configuration

Parameter	Value	Notes
<code>top_k_semantic</code>	5	Max vector results
<code>top_k_keyword</code>	5	Max BM25 results
<code>rrf_k</code>	60	RRF smoothing constant (authoritative; read by <code>hybrid_search.py</code>)
<code>max_context_tokens</code>	4000	Token budget for retrieved context (soul-preamble-aware; bounds LM Studio prompt)
<code>min_score</code>	0.028	RRF fused-rank threshold (NOT cosine sim — different scale; permissive by design)
<code>chunk_size</code>	512	Chars per chunk (sentence-snapped)
<code>chunk_overlap</code>	50	Overlap between adjacent chunks
<code>batch_size</code>	50	Embedding batch size (config); inference batch is 32
<code>vector_backend</code>	chroma	'chroma' (default) or 'pgvector' (opt-in)

E. Soul Governance

Identity ≠ memory ≠ topology. Soul is who the agent IS. Memory is what the agent KNOWS. Topology is what the agent CAN DO. Each is governed independently. v1.0.8 adds `soul_max_chars` (8000 char ceiling on the soul preamble prepended to every LLM prompt) to prevent an oversized `soul.md` from pushing the assembled prompt past the LM Studio context budget. All v1.3.0 hardening is preserved.

Soul Governance Mechanisms

- **Drift Detection.** On every `PersonalityManager.__init__`, SHA-256 of `soul.md` is compared to the latest sha256 in `soul_versions`. Mismatch triggers forensic `audit_log` event AND automatic recovery version insert.
- **Atomic Evolution.** `apply_evolution` writes to `soul.md.tmp` then calls `os.replace()` — POSIX-atomic rename. No partial-write corruption.
- **`soul_max_chars` cap (NEW v1.0.8).** In-memory soul truncated to 8000 chars on load/apply with a warning; `soul.md` on disk left intact. Prevents context budget blow-out.
- **Automatic TTL Pruning.** `maintenance(ttl_days)` invoked from `__init__` using `interaction_ttl_days` (365d) so old interaction rows are cleaned on every gateway start.
- **Threading Lock.** `threading.Lock` serializes every SQLite write: drift recovery, version insert, interaction insert, TTL prune.
- **Optional Postgres backend.** Set `database_url` or `CYCLAW_DB_URL` env var to move soul DB off SQLite while keeping offline-first default.

Write-Order Invariant (Crash-Safe)

```
1. Write proposed soul to soul.md.tmp 2. os.replace(soul.md.tmp, soul.md) # POSIX-atomic rename 3. INSERT new
version row into SQLite (under threading.Lock) 4. Truncate in-memory soul_core to soul_max_chars (8000) 5.
audit_log({'event': 'soul_evolution_applied', ...})
```

Soul API Constraints

- GET /soul — read-only (current content + version + path)
- POST /soul/propose — generates (`current_sha`, `proposed_sha`) diff WITHOUT applying
- POST /soul/apply — requires human-authored reason string; no anonymous mutations; fail-closed when `CYCLAW_API_KEY` unset
- POST /soul/reload — re-reads `soul.md` after manual edit
- POST /soul/restore — restores `soul.md` from `.bak` backup
- No endpoint allows silent or automated soul rewrites

Personality Config

Parameter	Value	Description
<code>personality.enabled</code>	<code>true</code>	Toggle soul injection
<code>personality.soul_path</code>	<code>data/personality/soul.md</code>	File-as-truth
<code>personality.db_path</code>	<code>data/personality/cyclaw_soul.db</code>	Shadow DB (SQLite default; Postgres opt-in)
<code>personality.interaction_ttl_days</code>	<code>365</code>	Prune old interaction logs
<code>personality.soul_max_chars</code>	<code>8000</code>	Hard ceiling on soul preamble chars (NEW v1.0.8)

Parameter	Value	Description
personality.database_url	null	Optional Postgres backend — set or use CYCLAW_DB_URL env var

F. Model Layer

Model	Type	Endpoint	Gating
all-MiniLM-L6-v2	Embeddings (384d)	CPU local, .emb_cache/	Always available
qwen2.5-7b-instruct	Chat LLM (local)	127.0.0.1:1234/v1	LM Studio required
grok-4.3	Chat LLM (external)	api.x.ai/v1	Triple-gated; grok-4.3 is current (grok-4 retired 2026-05-15)

LocalLLMClient: Timeout 300s, temperature 0.3, max_tokens 3000. Bounded exp-backoff retry on transport/5xx/429 (not on read-timeouts — LM Studio stall signature). Context budget enforced: graph.py bounds prompt INPUT to max_context_tokens (4000 tokens) so the assembled prompt never triggers LM Studio's '0% processing' stall (requires LM Studio context >= 4000 + 3000 + 1500 headroom).

GrokClient — Triple Gate (defense in depth)

- **app.mode == 'hybrid'** (config flag)
- **models.grok.enabled == true** (config flag)
- **user_confirmed_online == true** (per-query, set by client resubmit)

All three must be simultaneously true. GrokClient validates each precondition defensively even though the graph topology already prevents invalid calls. **policy.fallback.send_local_context_to_grok = false** by default: retrieved docs are NOT forwarded to Grok unless explicitly opted in. grok_max_prompt_chars (8000 default) caps per-call token spend.

G. MCP Server

Property	Value
Protocol	JSON-RPC over stdio
Server Name	cyclaw-hybrid-rag v1.0.0
Tool	hybrid_search
Input schema	{ query: string, top_k?: int, mode?: 'hybrid' 'vector' 'bm25' }
Sampling	null — cannot invoke any LLM (retrieval-only by protocol design)
Write access	None (read-only against existing indices)

The sampling = null constraint is structural, not policy. The MCP server cannot autonomously invoke an LLM. Any LLM processing must go through the gateway's LangGraph controller with full audit coverage — the MCP path bypasses none of the five security invariants.

H. Security and Sanitization

v1.0.8 expands the prompt-injection sanitizer from 13 to **33 curated regex patterns** (categories: Core Override 8, Role Reassignment 6, System/New Instructions 6, Memory/Persistence Manipulation 6, Authority/Urgency 3, Tool/Action Hijacking 2, Light Obfuscation/Jailbreak 2). All patterns are compiled with `re.IGNORECASE` and use `\s+` for whitespace tolerance (corpus-safe). The sanitizer operates at two levels: user input (reject on match, raises `PromptInjectionError`) and corpus chunks at ingestion (strip + replace with `[FILTERED]`).

Banned Patterns — 33 Total (OWASP LLM01 + Agent Security)

Pattern (regex)	Category	Since
<code>ignore\s+(previous all prior)\s+instructions</code>	Core Override	v1.0.8
<code>ignore\s+all\s+previous</code>	Core Override	v1.0.8
<code>ignore\s+safety</code>	Core Override	v1.0.8
<code>disregard\s+(previous all prior)</code>	Core Override	v1.0.8
<code>forget\s+(previous all prior)\s+instructions</code>	Core Override	v1.0.8
<code>override\s+instructions</code>	Core Override	v1.0.8
<code>bypass\s+(safety filters restrictions rules)</code>	Core Override	v1.0.8
<code>do\s+anything\s+now</code>	Core Override	v1.0.8
<code>you\s+are\s+now</code>	Role Reassignment	v1.0.8
<code>you\s+are\s+no\s+longer</code>	Role Reassignment	v1.0.8
<code>pretend\s+(you\s+are to\s+be)</code>	Role Reassignment	v1.0.8
<code>act\s+as</code>	Role Reassignment	v1.0.8
<code>act\s+as\s+if\s+you\s+have\s+no\s+restrictions</code>	Role Reassignment	v1.0.8
<code>roleplay\s+as assume\s+the\s+role</code>	Role Reassignment	v1.0.8
<code>new\s+instructions\s*:</code>	System Instructions	v1.0.8
<code>system\s+prompt\s*:</code>	System Instructions	v1.0.8
<code>admin\s* developer\s+override</code>	System Instructions	v1.0.8
<code>sudo\s+mode root\s+access</code>	System Instructions	v1.0.8
<code>maintenance\s+mode.*safety\s+filters\s+disabled</code>	System Instructions	v1.0.8
<code>critical\s+update\s+to\s+your\s+instructions</code>	System Instructions	v1.0.8
<code>remember\s+(for\s+future this\s+for\s+later as\s+permanent in\s+memory)</code>	Memory Poisoning	v1.0.8
<code>save\s+this\s+(for\s+later in\s+your\s+memory permanently)</code>	Memory Poisoning	v1.0.8
<code>update\s+your\s+(memory knowledge\s+base soul)</code>	Memory Poisoning	v1.0.8
<code>store\s+this\s+as\s+(as\s+rule permanent core\s+instruction)</code>	Memory Poisoning	v1.0.8
<code>core\s+instruction,\s*never\s+forget</code>	Memory Poisoning	v1.0.8
<code>add\s+to\s+your\s+(knowledge memory personality)\s+base</code>	Memory Poisoning	v1.0.8
<code>critical\s+update important\s+update</code>	Authority/Urgency	v1.0.8
<code>from\s+(it\s+department system\s+administrator)</code>	Authority/Urgency	v1.0.8

Pattern (regex)	Category	Since
urgent action\s+required	Authority/Urgency	v1.0.8
execute\s+command run\s+this\s+code	Tool Hijacking	v1.0.8
call\s+function use\s+tools+to invoke\s+tool	Tool Hijacking	v1.0.8
jailbreak dan\s+mode developer\s+mode	Obfuscation	v1.0.8
base64 decode\s+and\s+execute	Obfuscation	v1.0.8

PII Redaction (audit logger)

Type	Pattern	Replacement
Emails	Standard email regex	[REDACTED_EMAIL]
IPv4	Dot-notation	[REDACTED_IP]
Bearer tokens	Bearer\s+[A-Za-z0-9\-_\.]+\s*	[REDACTED_SECRET] (NEW v1.0.8)
API keys	api[_-]?key["\s]*[:=]["\s]*[\w]{4,}	[REDACTED_SECRET] (NEW v1.0.8)
AWS Keys	AKIA[0-9A-Z]{16}	[REDACTED_SECRET]
Slack Tokens	xox[baprs]-[0-9a-zA-Z-]+\s*	[REDACTED_SECRET]
GitHub PATs	ghp_[a-zA-Z0-9]{36}	[REDACTED_SECRET]
OpenAI keys	sk-[a-zA-Z0-9]{32,}	[REDACTED_SECRET]

TrustedHostMiddleware (NEW v1.0.8)

Host header checked against allowed_hosts allowlist before any handler runs (HTTP 400 on violation). Configured in config.yaml security.allowed_hosts. Default: 127.0.0.1, localhost, and any configured LAN host (e.g. 10.0.0.112 for home-lab). Mitigates DNS-rebinding attacks.

I. Browser UI

Both UIs are served via the StaticFiles mount on the gateway and run fully against `http://127.0.0.1:8787` — no external JS dependencies, no CDN-loaded scripts.

- **terminal.html (~49 KB)** — primary Soul Console: query input → /query, real-time answer + sources + full RRF retrieval metadata (semantic_score, keyword_score, rrf_score, rrf_semantic_contrib, rrf_keyword_contrib), Soul panel for load/propose/apply/reload/restore
- **extractor.html** — corpus extraction utility (carried over from v1.2.0)

J. Agentic Layer (v0.1 — Experimental)

The agentic layer is strictly out-of-band — it runs only via `python -m agentic.cli` and is **never imported by `gate.py`, `graph.py`, or `mcp_hybrid_server.py`**. This isolation preserves all five security invariants. v0.1 supports read-only GitHub operations. Write operations are a dry-run scaffold: even in write mode v0.1 only produces plans and never mutates GitHub.

Agentic Configuration

Parameter	Value	Notes
enabled	false	Master switch; CLI no-ops while false
repo	CGFixIT/CyClaw	Validated owner/name, no shell metacharacters
mode	'read'	'read' (safe default) 'write' (opt-in, still gated)
writes_enabled	false	Second flag; writes need mode=write AND this AND reason AND confirm
gh_timeout_sec	30	Wall-clock ceiling per gh subprocess
gh_retries	2	Extra attempts on transient gh failure; deterministic errors fail fast
allowed_read_ops	pr_view, pr_list, pr_diff, issue_view, issue_list, repo_view	Only these ops may run

Isolation Guarantees

- Never imported by `gate.py`, `graph.py`, or `mcp_hybrid_server.py` — out-of-band by architecture
- Reads use 'gh' CLI (argv-list, no shell, no token forwarded by CyClaw)
- Write ops require: mode=write AND writes_enabled=true AND human reason AND explicit confirm
- v0.1 write mode produces dry-run plans only — no actual GitHub mutations
- Registry path validated: under repo data/ tree, no shell metacharacters

K. Connectors (v0.1 Scaffolds)

Filesystem Connector

Scoped read/write connector for local filesystems. Runs only via **python -m agentic.fsconnect.cli** — never imported by gateway or graph. READS are scoped to `allowed_roots` (read-only). WRITES go only to `writable_roots`, are gate-disabled by default, and require `writes_enabled` + human reason + confirm.

Parameter	Value	Notes
enabled	false	Master switch; CLI no-ops while false
allowed_roots	[]	Required when enabled; each must resolve to existing dir
allowed_fs_ops	fs_list, fs_stat, fs_read, fs_grep, fs_glob	Operator-configurable read subset
follow_symlinks	false	Hard default; ANY symlink under a root is DENIED
max_file_bytes	5 MiB	Read cap enforced via fstat on open handle
writes_enabled	false	Master write switch; false → dry-run plans only
writable_roots	[null]	null = OS default (~/.CyClaw-FS on Linux)
max_write_bytes	10 MiB	Write cap
scan_content	true	Advisory OWASP+banned_patterns flagging on read content
index_enabled	false	Toggle RAG-corpus indexing of index_root

SQL Connector (Disabled Scaffold)

Read-only on-prem SQL adapter (Postgres / MSSQL). Fully disabled scaffold — runs only via **python -m agentic.sqlconnect.cli**. Read-only enforced at the session level (SET TRANSACTION READ ONLY). DSN read from env var only (CYCLAW_SQL_DSN) — never hardcoded.

Parameter	Value	Notes
enabled	false	Off by default; pure no-op
driver	'postgres'	'postgres' 'mssql'
dsn_env	CYCLAW_SQL_DSN	DSN from env var only
read_only	true	Hard: SET TRANSACTION READ ONLY
max_rows	1000	Row cap — no full-table materializations
statement_timeout_ms	5000	Query timeout
allowed_sql_ops	schema_list, table_preview, run_select, explain, row_count	Read-only ops

Dropbox Corpus Sync

rclone-based corpus pull (bisync opt-in). Runs only via **python -m sync.cli** — never imported by gate.py or graph.py. Dropbox refresh token lives in `rclone.conf` only, never in `config.yaml`.

Parameter	Value	Notes
enabled	false	Off by default; sync CLI no-ops while false

Parameter	Value	Notes
direction	'pull'	'pull' (safe default) 'bisync' (opt-in, discouraged)
max_delete	20	Safety fuse: abort if > N deletions
max_transfer	'1G'	Safety fuse: abort if run would transfer > this
checksum	true	rclone --checksum (hash compare, not mtime)
reindex_on_change	true	Exit 10 when corpus changed — triggers re-indexer
auto_reindex	false	When true: sync CLI runs indexer itself on change
include_soul	false	data/personality/ is NOT sync-safe — never include
sync_timeout_sec	3600	Wall-clock ceiling on rclone run

L. NeMo Guardrails (v0.1 Skeleton)

Disabled-by-default defense-in-depth layer. Runs strictly out-of-band via `python -m guardrails.cli` — **never imported by `gate.py`, `graph.py`, or `mcp_hybrid_server.py`**. This isolation preserves the five security invariants; the graph topology stays the sole routing/policy source. `nemoguardrails` is an OPTIONAL dependency: the layer soft-imports it and degrades to offline heuristic rails (injection + soul-mutation block + token-overlap grounding) when `nemoguardrails` is absent.

Rails Configuration

Parameter	Value	Notes
engine	openai	LM Studio / Ollama expose OpenAI-compatible API
base_url	127.0.0.1:1234/v1	Loopback LM Studio; offline-only
model	qwen2.5-7b-instruct	Keep in sync with <code>models.local_llm.model</code>
hallucination_threshold	0.18	[0..1] token-overlap floor; below = ungrounded
block_message	(configurable)	Shown when a rail blocks a request

Rails Taxonomy

Rail Type	Flows	Trigger
Input	check_injection, check_jailbreak, check_soul_mutation	Run on every inbound query
Output	check_grounding, check_soul_leak	Run on LLM answer before return
Topical	stay_in_local_knowledge, no_unauthed_external_advice	Boundary enforcement for soul/personality topics

M. Test Suite

All tests use mocked external dependencies — no live LM Studio / Chroma / Grok required. CI runs pytest with --tb=short -q and enforces ruff lint + mypy type-check + bandit security gates.

File	Coverage
tests/conftest.py	Shared fixtures + TEST_CONFIG mirroring prod
tests/test_graph.py	All 7 LangGraph paths: local, grok, offline, gate, error; partial-injection node test
tests/test_gate.py	/query, /health, /soul/*, /audit/summary, /ops/sync, /ops/agentric, error responses
tests/test_personality.py	PersonalityManager: init, propose, apply, reload, soul_max_chars truncation
tests/test_sanitizer.py	All 33 banned patterns, max_input_chars, corpus sanitization
tests/test_audit.py	SHA-256 hashing, PII redaction (incl. Bearer+api_key v1.0.8), JSONL output
tests/test_hybrid_search.py	Semantic, keyword, RRF fusion, graceful degradation, pgvector backend
tests/test_stemmer.py	Plurals, verb forms, technical AI/DevOps terms, custom stems
tests/apipsTest.ps1	PowerShell API smoke tests
tests/test_rate_limit.py	Sliding window, expiry, per-IP isolation, SQLite/Postgres persistence
tests/test_personality_changes.py	Drift detection, atomic apply, TTL prune, version ordering
tests/test_mcp_server.py	MCP protocol validation, sampling=null invariant
tests/test_security.py	BM25 JSON (not pickle), API key auth (Bearer), logging setup, TrustedHostMiddleware
tests/test_telemetry_kill.py	All 10 LangChain/Chroma/OTel env var kills verified at startup
tests/test_conftest_fixtures.py	Shared mock fixtures for retriever, LLM, config
tests/TEST_SUITE_AUDIT.md	Test coverage audit and gap analysis

Pre-flight test runner:

```
# Targeted pre-flight (security + telemetry + personality + rate) python -m pytest tests/test_security.py
tests/test_telemetry_kill.py \ tests/test_personality_changes.py tests/test_rate_limit.py -q # Full suite python
-m pytest tests/ -q
```

N. Deployment & Validation

Install / Quick Start (uv preferred)

```
# Preferred: uv (hermetic, reproducible) pip install uv uv pip install -r requirements.txt -c constraints.txt # Or
legacy pip: pip install torch==2.12.1+cpu --index-url https://download.pytorch.org/whl/cpu pip install -r
requirements.txt -c constraints.txt # constraints.txt pins the full transitive dependency tree. # Always use -c
constraints.txt for production installs. # ChromaDB note: PersistentClient (offline, embedded mode only). # NO
HttpClient. NO trust_remote_code. CVE-2026-45829 risk accepted for local/offline use. # See SECURITY.md for
justification. # Offline note: run indexer once on networked machine to cache all-MiniLM-L6-v2 # then set
HF_HUB_OFFLINE=1 for fully air-gapped operation.
```

v1.0.8 Upgrade Path

1. Pull latest main: **git pull origin main**
2. Install updated deps: **uv pip install -r requirements.txt -c constraints.txt**
3. Delete index/ and rebuild: **python -m retrieval.indexer** (BM25 now JSON, not pickle; pgvector opt-in available)
4. Copy env vars from cyclaw_telemetry_kill.env if not already set
5. Run pre-flight: **python -m pytest tests/test_security.py tests/test_telemetry_kill.py -q**
6. Start gateway: **uvicorn gate:app --host 127.0.0.1 --port 8787** (or: **cyclaw-server** if installed via pyproject)
7. Smoke test: **curl -X POST http://127.0.0.1:8787/query -d '{"query":"what is CyClaw"}' -H 'Content-Type: application/json'**
8. Verify health: **curl http://127.0.0.1:8787/health** shows all deps healthy, mode=offline, graph_ready=true

Rollback Plan

Stop the gateway, reinstall deps from the prior requirements.txt (without -c constraints.txt), rebuild the index (python -m retrieval.indexer), restart. The cyclaw_soul.db schema is forward-compatible — no data loss. The BM25 index format changed from pickle (.pkl) to JSON (.json) — the old pickle file will be regenerated automatically on first indexer run.

O. v1.0.8 Change Log (vs. v1.4.0 baseline)

Agentic Layer (New)

- agentic/cli.py — read-only GitHub ops (pr_view, pr_list, pr_diff, issue_view, issue_list, repo_view)
- Write scaffold: dry-run plans only; no GitHub mutations in v0.1
- skills_registry.json support for operator-defined read ops
- gh CLI subprocess (argv-list, no shell, no token forwarded by CyClaw)
- Fully isolated: never imported by gate.py, graph.py, mcp_hybrid_server.py

Filesystem & SQL Connectors (New)

- agentic/fsconnect/ — scoped read/write; content-safety scan; RAG index of file share
- agentic/sqlconnect/ — read-only Postgres/MSSQL; disabled scaffold; DSN from env var only

Dropbox Corpus Sync (New)

- sync/cli.py — rclone pull (bisync opt-in); safety fuses: max_delete=20, max_transfer=1G
- auto_reindex toggle; exit-10 on corpus change for cron-chain pattern
- Dropbox token lives in rclone.conf only — never in config.yaml

NeMo Guardrails (New)

- guardrails/cli.py — input/output/topical rails; nemoguardrails soft-import; offline heuristic fallback
- Fully out-of-band; never imported by gateway or graph

Security Hardening

- Prompt filter expanded: 13 → 33 patterns (memory poisoning, authority/urgency, tool hijacking, obfuscation)
- PII redaction: Bearer tokens + api_key forms added to audit-log redaction (closes token-leak-to-audit gap)
- TrustedHostMiddleware: DNS-rebinding defense via Host header allowlist
- BM25 index format: pickle → JSON (eliminates pickle deserialization attack surface)
- API auth: CYCLAW_API_KEY fail-closed (401 if unset); constant-time hmac.compare_digest

Gateway & Graph

- Graph timeout: asyncio.wait_for wraps compiled_graph.invoke; HTTP 504 on exceed
- Rate limiter: optional SQLite/Postgres persistence for per-IP counters across restarts
- soul_max_chars: 8000 char ceiling on soul preamble; prevents LM Studio context budget blow-out
- All node deps injected at build time via functools.partial (stateless, safe to reuse)
- /audit/summary endpoint added (Bearer-gated); /ops/sync and /ops/agentic (Bearer-gated ops triggers)

Model Updates

- Grok model: grok-beta → grok-4.3 (grok-4 retired by xAI 2026-05-15)
- LLM timeouts: 720s → 300s; max_tokens: 5000 → 3000 (avoids LM Studio '0% processing' stall)
- max_context_tokens: 5000 → 4000 tokens (matches tighter output budget)
- grok_max_prompt_chars: 8000 char ceiling on Grok prompt (cost guard)

- pgvector opt-in: install `cyclaw[pgvector]` extra; ChromaDB remains default

Packaging

- pyproject.toml replaces setup.py/setup.cfg as canonical packaging spec
- cyclaw-server entry point: `uvicorn gate:app --host 127.0.0.1 --port 8787`
- uv preferred for hermetic installs; constraints.txt covers full transitive tree

P. Security Invariants (5 Topology-Enforced Guarantees)

These five invariants are structural properties of the system — enforced by architecture and graph topology, not by prompt instructions, code discipline, or operator trust. They are preserved and structurally strengthened in every v1.0.8 change.

1. RAG-First Retrieval

Retrieval node is the unconditional first node in every graph execution. No LLM call occurs without a prior retrieval attempt. This is structural: the graph entry point is 'retrieve' and there is no path that bypasses it.

Implementation anchor: `graph.set_entry_point('retrieve')`

2. Topology = Enforcement

Routing decisions (local_llm vs user_gate vs grok_fallback vs offline_best_effort) are made exclusively by graph edges and conditional nodes — never by LLM interpretation of instructions. `graph.add_conditional_edges()` with `score_router` / `user_gate_router` encode policy as structure.

Implementation anchor: `add_conditional_edges('route_by_score', score_router, {...})`

3. Identity ≠ Memory ≠ Topology

Soul (who the agent IS), memory/knowledge (what it KNOWS), and graph topology (what it CAN DO) are governed by independent subsystems with explicit boundaries and no cross-layer silent mutation. Soul evolution requires explicit human confirmation via `/soul/apply`. No graph node can modify soul.

Implementation anchor: `utils/personality.py` (soul) $\leftrightarrow$ `graph.py` (topology) – no shared mutation path

4. Audit Convergence

Every execution path terminates at `audit_logger` before returning to the gateway. There is no bypass. SHA-256 query hashing and PII redaction (including Bearer tokens and `api_key` patterns) are mandatory and structural — not opt-in.

Implementation anchor: `graph.add_edge('local_llm', 'audit_logger')` # all paths converge

5. Human-in-the-Loop for Evolution

Soul mutations require an explicit human-authored reason string via `POST /soul/apply`. No anonymous or automated self-rewrites are possible via any endpoint or graph path. The endpoint is fail-closed when `CYCLAW_API_KEY` is unset (HTTP 401). No graph node has write access to soul.

Implementation anchor: `require_api_key + hmac.compare_digest + human reason string (non-empty)`

All content derived from CyClaw v1.0.8 production codebase and architecture reference (main branch, commit 91cecb+, June 2026). This document reflects the actual running codebase verified via live sandbox testing on 2026-06-29. Five security invariants preserved and structurally strengthened.
github.com/CGFixIT/CyClaw